

Lab exercises

Part 07 — Agentic AI Systems · 07-agentic-ai/lab/EXERCISES.md

Lab — Agentic AI Systems

Work top to bottom. Each exercise has a **stated deliverable**; if you can't produce it, you haven't finished. No API keys are required — everything runs offline against the code in `../code/` or against your own reasoning on paper.

1. Place five real systems on the autonomy spectrum (45 min)

For each system below, name the point on the spectrum from notes/01 section 1 (single call, tool-using agent, multi-agent, autonomous long-running) it *should* occupy, and the point you suspect it's often *actually* built at because autonomy was maximized reflexively:

1. A tool that drafts a reply to an inbound sales email.
2. A system that reconciles a company's bank statement against its accounting ledger every night.
3. A "research assistant" that answers "what changed in our competitor's pricing this quarter."
4. A coding agent asked to fix one specific failing unit test.
5. A system that monitors production error logs and files tickets for new error signatures.

Deliverable: a 5-row table (system | should-be point | commonly-overbuilt-to point | one sentence why).

2. Run and interrogate the topology simulator (1.5h)

Run `python code/topology_simulator.py`.

- **2a.** Without changing any code, explain in 2-3 sentences why `hierarchical` costs more tokens than `supervisor` for the identical underlying task.
- **2b.** Change `BASE_ERROR_RATE` from `0.12` to `0.30` and rerun. Which topology's success rate advantage over `single-agent` grows the most, and why does that match debate/vote's theoretical purpose (notes/01 section 2.4)?
- **2c.** Change `LOOP_PROBABILITY` to `0.0` and rerun. Quantify exactly how many percentage points of `peer-handoff`'s success rate were being lost to hand-off loops alone, holding worker accuracy constant.
- **2d.** Add a **new** topology function: a supervisor that spins up only 2 workers (not 3) and has the supervisor itself answer the third sub-question directly, no hand-off. Add it to `TOPOLOGIES` and report where it lands in the token/latency/success table relative to the existing six.

Checks: your 2d topology's token count is between `single-agent` and `supervisor`. If it isn't, you implemented the setup or hand-off costs identically to the full 3-worker case somewhere.

3. Extend the hand-off information-loss demo (1.5h)

Run `python code/handoff_information_loss.py`. Read the final side-by-side table.

- **3a.** Write a **second** ticket (own scenario: a warranty claim, an infra incident, a contract dispute) with a critical fact buried in the third paragraph, in the same style as `TICKET`. Run all three hand-off strategies against it by adapting the script and report the same side-by-side table.
- **3b.** Design a keyword list for your new ticket that — like the one in the module — is built from the *symptom* rather than the *diagnostic history*, and confirm it drops the critical fact (matching the module's finding, not accidentally preserving it — this is easy to get backwards, watch for it).
- **3c.** Now write a hand-off strategy that **would** have preserved the fact — not "full ticket," but a genuinely better summarization heuristic (e.g., "always keep the last sentence of every paragraph" or "always keep sentences containing a negation like 'didn't help'"). Show it preserves the fact on both your ticket and the module's original one.

Deliverable: your ticket text, the side-by-side table, and your improved heuristic with justification for why it generalizes rather than being reverse-engineered to fit these two examples.

4. Topology selection under constraints (1h)

For each scenario, pick **one** topology from notes/01 section 2 and defend it in ≤ 5 sentences using the "problem shape it fits" column. The trap in at least two of them is picking a fancier topology than the task needs:

1. Summarize five independent analyst reports on the same company into one memo.
2. A user asks a coding assistant to "add a login page" — needs planning, file edits, and running tests, in an order that depends on what earlier steps find.
3. Answer one high-stakes medical-triage question where being wrong is costly and you have three independently-trained model checkpoints available.
4. Convert a batch of scanned invoices into structured JSON, always the same five extraction+validation steps in the same order.
5. Route an inbound support ticket to the right specialist team, where "right" sometimes only becomes clear after the first specialist starts reading it.

Checks: you picked pipeline for #4 and debate/vote for #3. If you picked supervisor for #4, re-read notes/01 section 2.5 — a fixed order with no runtime branching doesn't need a router.

5. HITL vs. HOTL trigger design (1h)

Take scenario 2 from Exercise 4 (the coding assistant). List every distinct *action* it might take (read a file, edit a file, run tests, install a dependency, push a commit, delete a file, call an external API). For each action,

assign HITL or HOTL and write one concrete trigger condition from notes/02 section 4 — not "use judgment," an actual threshold or event.

Deliverable: a table (action | posture | trigger condition). At least one action must be gated differently from the others — if every action gets the same posture, redo it; that's the mistake the notes warn against directly.

6. The decision framework, applied cold (1h)

Without re-reading notes/02 section 7 first, run these three requests through your own memory of the framework, then check yourself against the actual flowchart:

1. "Read this résumé and tell me if the candidate has 5+ years of Python experience."
2. "Every morning, pull yesterday's sales figures, compare them to forecast, and email a one-paragraph summary to the sales lead — if the miss is over 15%, also loop in their manager."
3. "Investigate why our checkout conversion rate dropped this week — pull data from three different internal systems, form a hypothesis, and check it against a fourth source before reporting back."

Deliverable: for each, your classification (single call / pipeline+LLM step / single agent / multi-agent), which framework question triggered your "stop" at that answer, and — for whichever ones you initially got wrong before checking — one sentence on what made you over- or under-scope it.

7. Stretch: quantify your own multi-agent decision (2h)

Pick a real or realistic automation idea from your own work. Using `code/topology_simulator.py` as a template (reuse its token-cost constants, or replace them with your own estimates of your target model's pricing), build a small script that compares your idea's single-agent cost against its proposed multi-agent cost, using YOUR task's actual number of sub-tasks and a realistic error rate.

Checks: your script prints a token multiplier. State explicitly whether that multiplier stays under roughly 15x (Anthropic's reported ceiling for a system where the cost was worth it — see papers/PAPERS.md [07-agentic-ai/papers/PAPERS.md]) and whether your task's value justifies whatever multiplier you computed, using the checklist from notes/02 section 1's "real reason to split" table — not vibes.